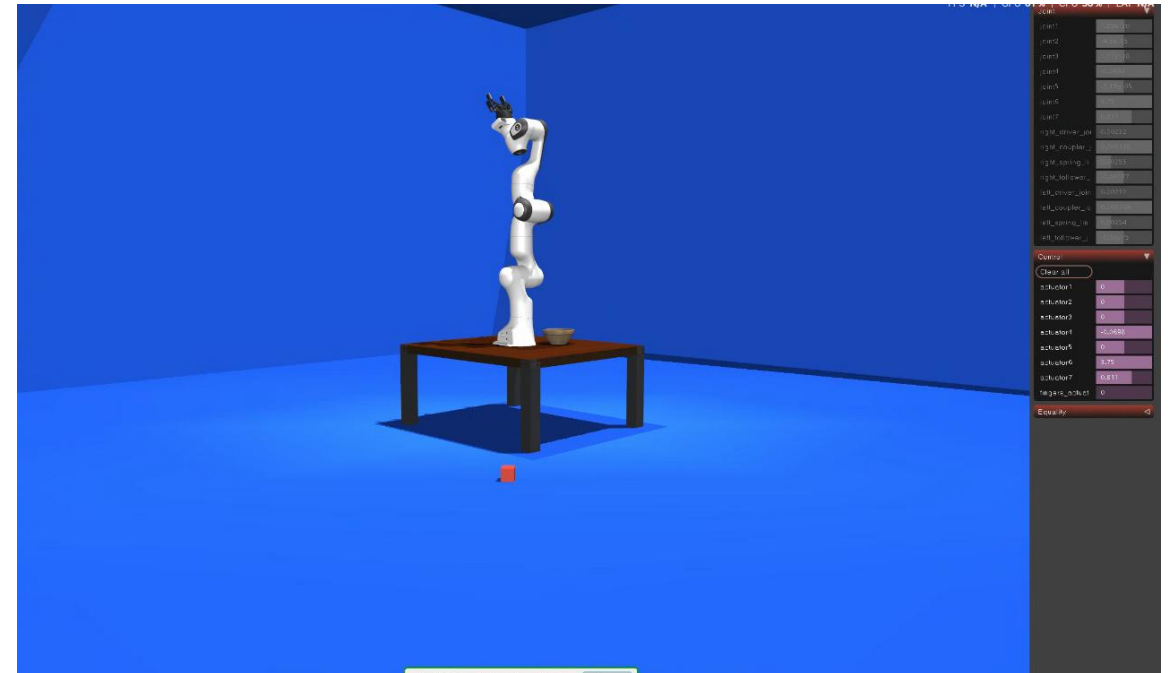


Robot arm learns how to make a bank shot

By Zeest Fatima

Initial Conditions

- Robot arm sits on a table in the middle of a square-shaped room.
- Room has dimensions 6m x 6m.
- Robot starts with ball gripped in hand.
- Walls all around are bounceable, including floor, roof and table .



Workflow Summary

1. Using PPO, robot learns to bounce a ball against any wall and land it on the floor, without hitting the table, floors or roof first.
2. Good PPO checkpoints are then used to collect trajectories/ episodes where the task was completed successfully.
3. A KNN regression model is used to map release parameters (i.e ball position, linear velocity and angular velocity at release) to final landing position on floor.
4. Pre-train a supervised imitation learning model using PPO trajectories to map actuator commands (7 joints + 1 gripper) to ball position, velocity and angular velocity at release.
5. Train a TD3 model online using the pre-trained model.

PPO Training

- Task: ball is released from gripper -> hits wall -> hits floor. It must not hit floor, table or roof before wall.
- Rewards: Sparse

Active Reward Terms		
Index	Name	Weight
0	ball_movement	6.0
1	early_gripper_open	0.2
2	early_joint_velocity	0.08
3	ball_wall_without_table_or_ground_first	12.0
4	ball_random_target_wall_without_bad_surface_first	20.0
5	ball_landed_after_wall	15.0
6	gripper_not_opened_by_2s	-1.0
7	ball_hit_table_penalty	-1.0
8	ball_hit_ground_penalty	-1.0
9	ball_hit_roof_penalty	-1.0

Active Observation Terms in Group: 'actor' (shape: (47,))		
Index	Name	Shape
0	joint_pos	(15,)
1	joint_vel	(15,)
2	ee_to_ball	(3,)
3	ball_to_bowl	(3,)
4	bowl_position	(3,)
5	actions	(8,)

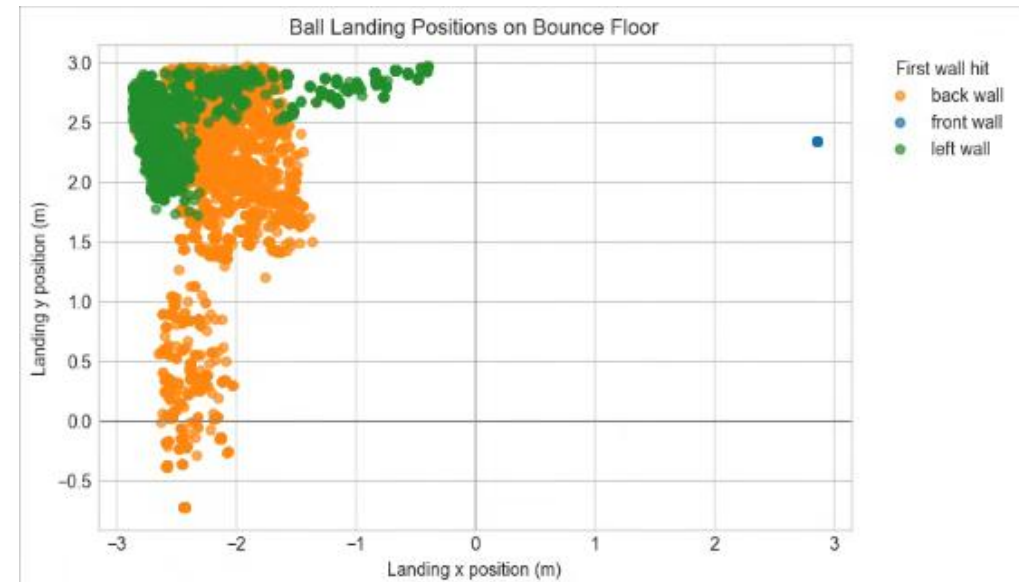
Active Observation Terms in Group: 'critic' (shape: (47,))		
Index	Name	Shape
0	joint_pos	(15,)
1	joint_vel	(15,)
2	ee_to_ball	(3,)
3	ball_to_bowl	(3,)
4	bowl_position	(3,)
5	actions	(8,)

PPO Data Collection

- Training ran until 5000 checkpoints, and checkpoints saved at intervals of 500.
- Best checkpoints seem to be between 500 and 3000. Collect 1000 successful trajectories from 23 best checkpoints. In total collect 23000 “good” + 460 “bad” trajectories.



Floor landing positions of “good” trajectories by checkpoint.



Floor landing positions compared to which wall the ball first hit.

PPO Data Collection

- Metrics Collected Summary:
 - **Per-step data:** action, reward, achieved goal, desired goal, ball position + velocity, episode ID + step, timestep
 - **Timing:** wall contact step, landing step, ball release step
 - **Release state:** gripper position at release + ball position, velocity and angular velocity (spin) at release
 - **Landing + wall contacts:** wall contact position, final landing position, Boolean: hit wall first, landed after wall, hit ground OR table OR roof first, is it success
 - **Checkpoint meta data:** good eps + bad eps per checkpoint, checkpoint ID etc., action + goal + observation dimensions

Ball Release State

- Ball release metrics are defined by the timestamp at which:
 - 1) $\text{abs}(\text{ball height} - \text{robot palm height}) > 0.02\text{m}$ **AND**
 - 2) $\text{abs}(\text{ball linear velocity} - \text{robot palm velocity}) > 0.05 \text{ m/s}$
- At the first timestep where release became true, the collector saved:
 - Release step
 - Ball position
 - Ball velocity
 - Ball angular velocity
 - Gripper palm position at release

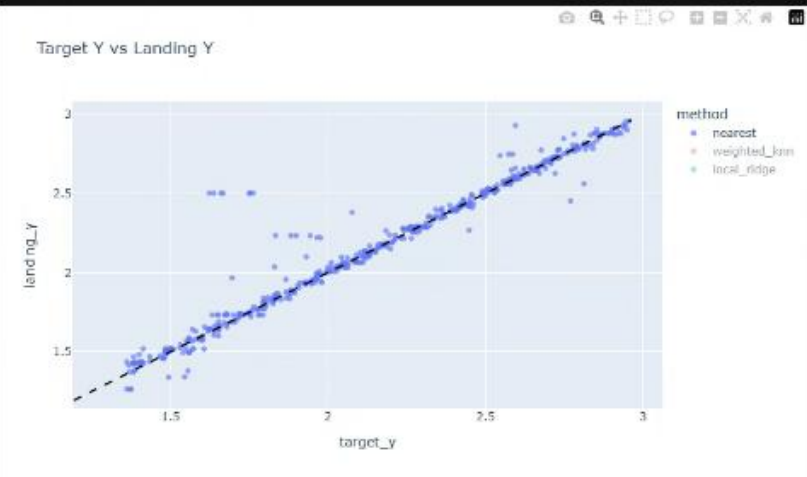
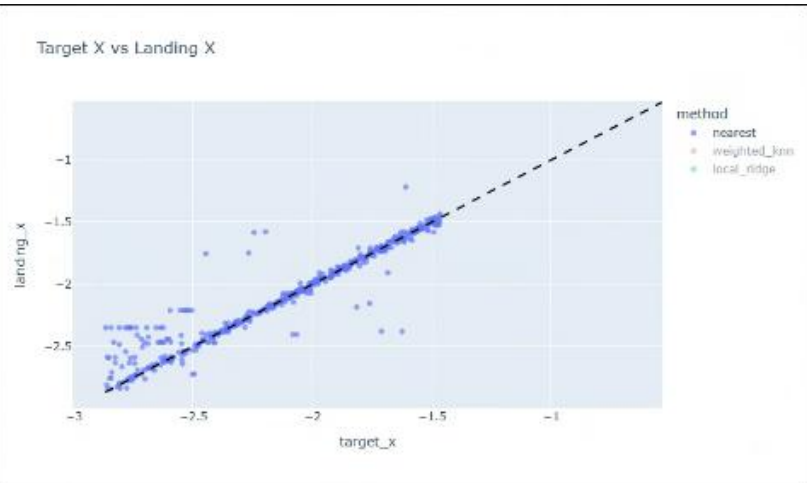
K-Nearest Neighbors Regression

- Task: Mapping release parameters (the ball's position, velocity and spin) to the final landing point on the floor, using data from PPO.
- Tested 3 regression models:
 - **Nearest Neighbor regression:**
Given a target landing point, it finds the one PPO trajectory whose landing point is closest to the target. Then it reuses that trajectory's release state.
 - **Weighted KNN:**
This finds several nearby PPO landings, not just one. Then it averages their release states, with closer examples getting more weight.
 - **Local Ridge Regression**
For each target landing point, it still finds nearby PPO examples. But instead of averaging their release states, it fits a small local linear model.

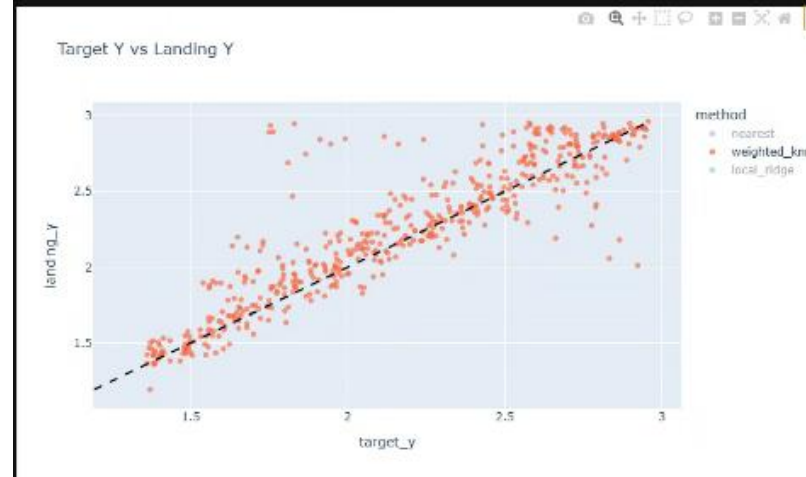
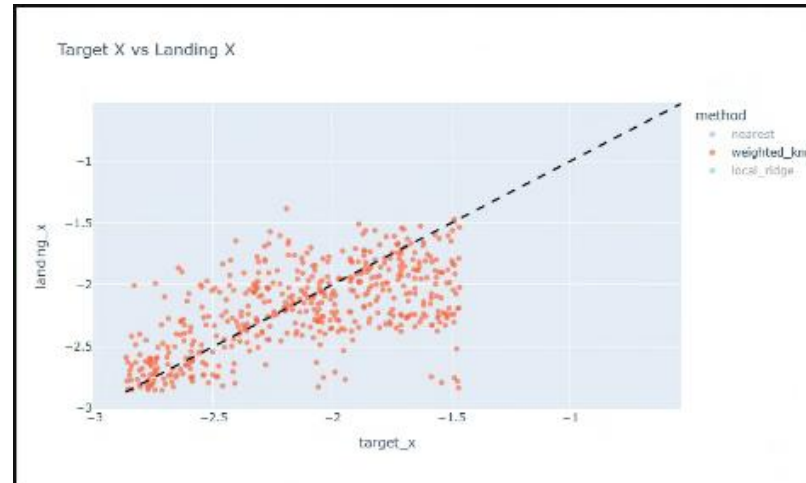
KNN Evaluation

- Tests whether release state produces desired landing position when simulated.
- In the evaluation, the robot arm does not throw the ball, instead the evaluator directly sets the ball's position, linear velocity and angular velocity (spin) to the predicted release state, then steps MuJoCo simulation forward with neutral gripper actions.
- Evaluate KNN release planner and simulation physics but not actuator control.
- Finally, we plot target x and y coordinates (from PPO) against actual landing coordinates (from MuJoCo simulation).

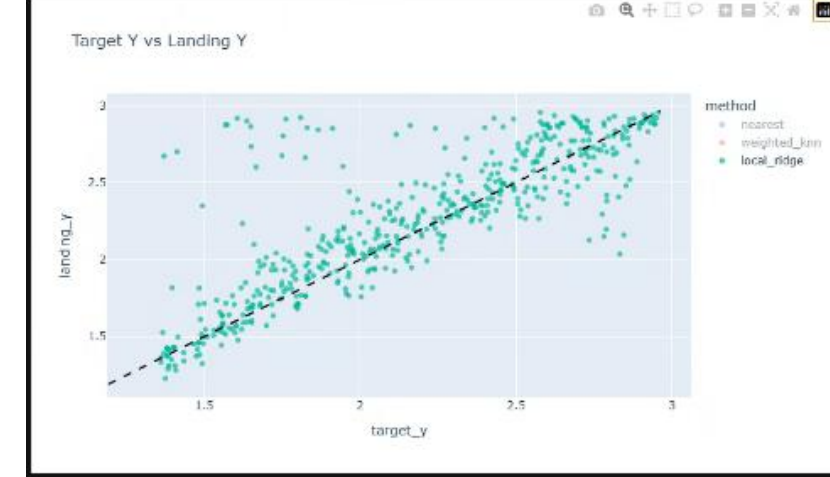
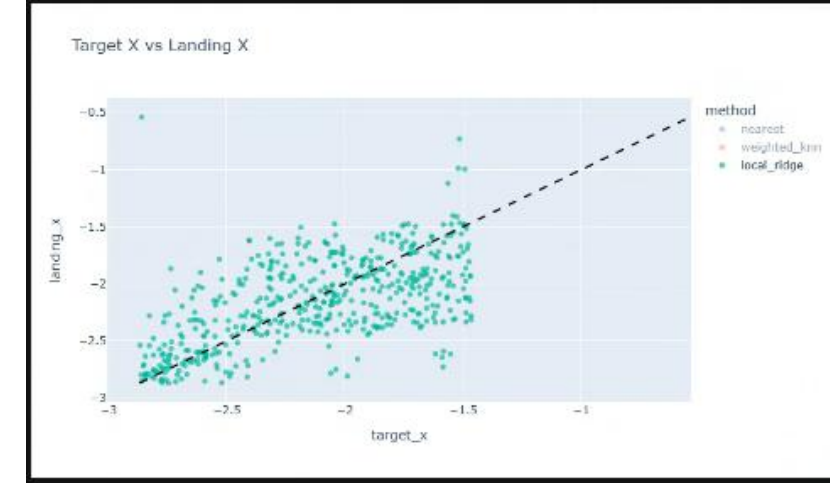
KNN Model Performance Comparison



Simple K-Nearest Neighbors
80% success rate



vs
Weighted KNN
13% success rate



vs
Local Ridge regression,
13% success rate

Offline Supervised Imitation Learning Model

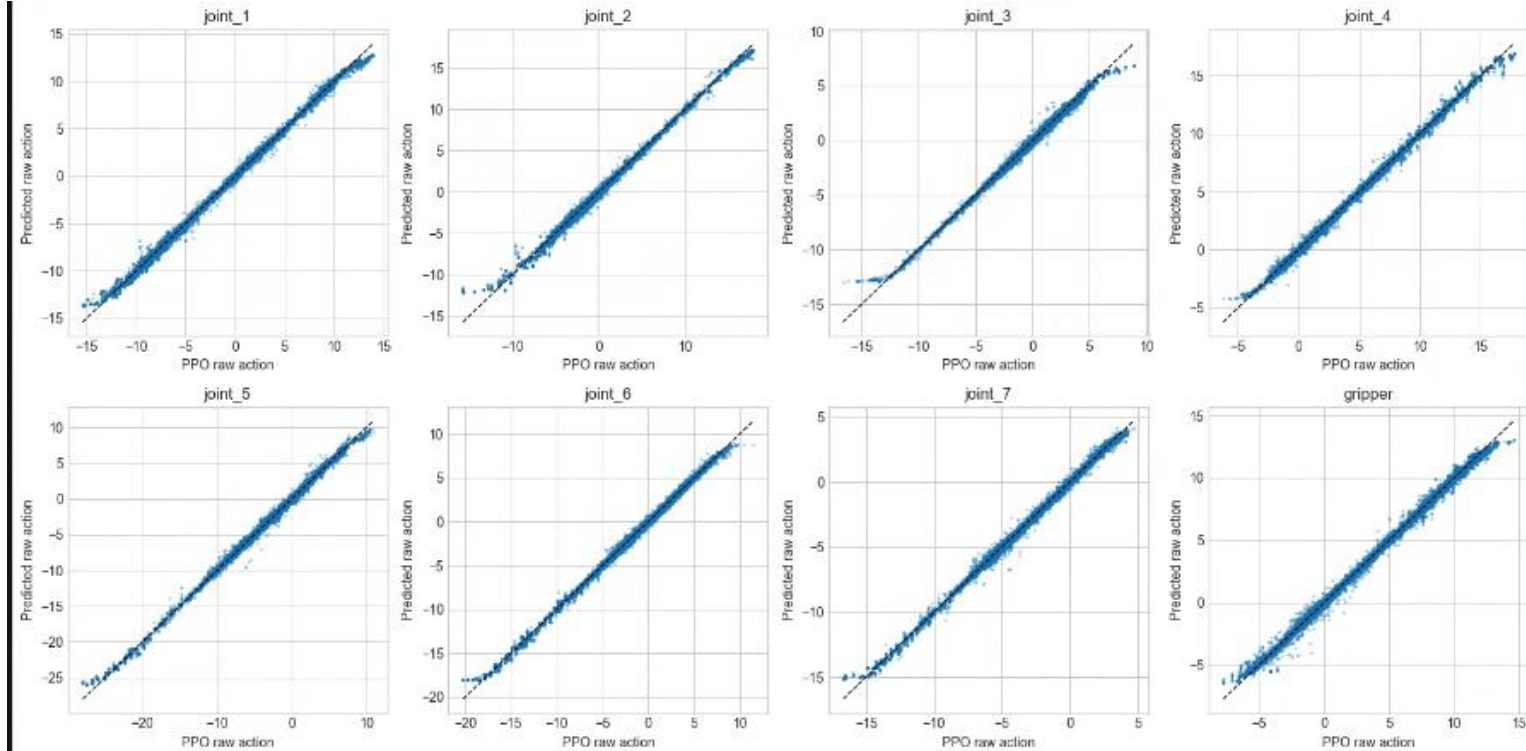
- Offline pre-training for TD3
- Task: Learns to map actions of 8 actuator joints (including gripper) to ball's release state (release position, velocity, spin).
- Trained on all 23000 checkpoint episodes (only successful ones).
- Uses ideas from a paper on max-achievement/ release-match scoring. Optimize weighted action prediction error, so we use weighted MSE imitation loss, rather than cumulative reward maximization.
- Predicted action = actor(observation, desired release state)
- Loss = MSE(Predicted action , PPO action)

Offline Supervised Imitation Learning Model

- Learns which PPO timesteps matter more. Used the following dense rewards:
 - **release position closeness score:** $\exp(-\text{abs}(\text{ball pos} - \text{target release pos}) / \text{pos scale})$
 - **release velocity closeness score:** $\exp(-\text{abs}(\text{ball vel} - \text{target release vel}) / \text{vel scale})$
 - **release spin closeness**
 - **Release-match score:** $\text{pos weight} * \text{pos score} + \text{vel weight} * \text{vel score}$
 - **best score reached so far:** for each eps, computes best release-match score up to current timestamp.
 - **improvement in best score:** computes how much best score improved at each timestep.
 - **release timestep / near-release steps:** treated small window around release with higher weight
- **Final weight:** CLIP (0.10 + 1.5 * best score + 4.0 * improvement + 1.0 * near release, 0.05 , 8.0)

Offline Supervised Imitation Learning Model

- Trained on all of 23000 episodes collected from PPO
- The actor predicts/ outputs normalized actions. We denormalize and compare the action's joint movement predictions to PPO actions, and get the following plots.
- Seems to be imitating PPO trajectories well.



TD3 training

- Task: improve actuator policy beyond offline behavior cloning.
- Replay split into 3 buffers:
 - Offline seed replay
 - Sample ratio: 20%
 - Filled with successful PPO data before online training starts
 - Target goal is PPO recorded release state (vel + pos + spin at release)
 - Online replay
 - Sample ratio: 50%
 - Actor is given a sampled desired release state (from PPO release dataset). You also specify exploration noise.
 - Reward is max-achievement dense reward
 - HER replay
 - Sample ratio: 30%
 - At the end of each online episode, notebook looks at best release state the robot achieved. Then it copies that into HER replay and relabels goal = best achieved release state
- Dense rewards set up identical to the pre-trained model.
- Critic trained from Bellman targets and actor trained from critic estimates + pre-trained behavior cloning model estimates.